

A Comparative Performance Analysis of a Two Dimensional Diffusion Model Using Different Compiler Optimisation Techniques

Exam No. B024703, Group 03

November 14, 2016

1 Introduction

This report details experiments undertaken to compare the performance achieved using two different compilers, The GNU C Compiler and the PGI C Compiler, on a model using different terrain sizes. The compilers were used with different optimisation flags and results were gathered using various sizes of terrain.

2 Methods

2.1 The Source Code

The source consists of code written in C99. It also includes a suite of unit tests written using the CUnit framework. These tests were used to verify the stability of each compilation technique.

The code, called CLiPH or Circle of Life for Pumas and Hares, computes a diffusion model of predators and prey. It takes as an input a map which contains $M \cdot N$ elements, where M is height and N is width. Each of these elements is either land or sea. No predators or prey can exist in areas designated as sea. As an output it calculates an average population density for

both the predators and the prey as well as generating separate population density maps as PPM files for both classes of animal.

Among other things, the frequency with which these outputs are generated are defined by parameters stored in a configuration file. As the PPM output files can be large where N and M are also large, a value was selected in this configuration file so that these maps would never be output. This allows the performance analysis here to focus on compute time, rather than the I/O that would be involved in writing these large files to disk. However, the calculation of a mean population density involved some not inconsiderable computation. These densities were therefore calculated every 100 timesteps. The parameters were kept constant throughout all experiments.

2.2 Compilation

The source code was compiled using two different compilers. These were the GNU C Compiler and the PGI C compiler. The source was compiled on and for an Intel quad core Core-i5 CPU. Full CPU details and compiler versions are included as an appendix.

In both cases the source was compiled first with no optimisation flags, then with the `-O1` `-O2` and `-O3` flags. These flags specify differing degrees of compiler optimisation.

2.3 Input Maps

To create a map to use as input for the program, an image in black and white was created with the GNU Image Manipulation Program (GIMP). In this black represented land and white sea. For the sake of simplicity the image was a square where $M = N = 1000$. This image was then resized using the ImageMagick tool to sizes between 10% and 400% of the original, with increments of 10%. Finally the images were converted to a ASCII map format readable by the CLiPH program using a small utility that converts darker colours to 1 and lighter colours to 0. This resizing of the same image was done to ensure that the ratio of land to sea would remain constant between simulations.

2.4 Running Timed Simulations

The variously sized images were used as inputs for the different executables generated in the compilation part of the experiment. These were timed using a timing feature built into the CLiPH program.

For the sake of repeatability all stages of the experiment were conducted programmatically using a Bash script and the Make utility for compilation as well as gnuplot for displaying results. These scripts are available on request from the author and would make conducting the same experiments on different processors trivial.

3 Results

3.1 Compilation, Stability and Correctness

In all cases the code was compiled with no warnings or errors from the compiler and the suite of unit tests passed completely. Beyond this it is hard to compare with surety the accuracy of the results generated, as the initial population densities are generated pseudo-randomly. This means that accuracy cannot be verified by repetition. It was noted that credible results were generated in all cases, with populations of predator and prey oscillating in a slightly out of phase manner, as would be expected.

The fully passing unit tests along with somewhat unscientific inspection of the output population densities convinced the author that correctness should not be a factor when comparing compilers, or the compiler optimisation techniques, used on this code.

Both GNU and The Portland Group, the authors of the two compilers, warn that using high levels of compiler optimisation increase compilation times and memory usage [1][2]. However, as in all cases compilation times was imperceptible to a human user, it was considered unnecessary to accurately measure or discuss further compilation time or memory use.

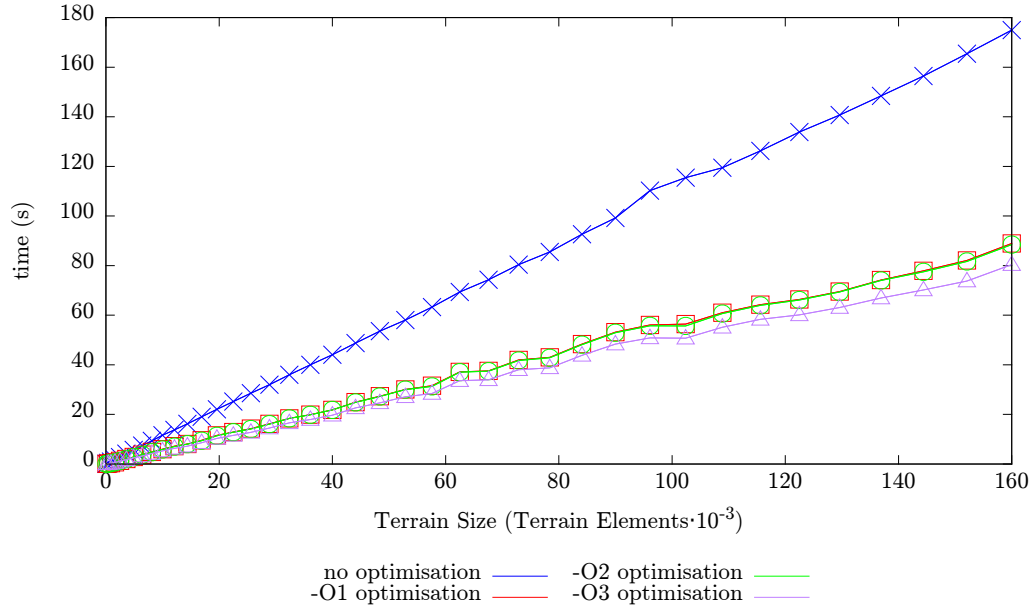


Figure 1: Graph showing GCC execution times

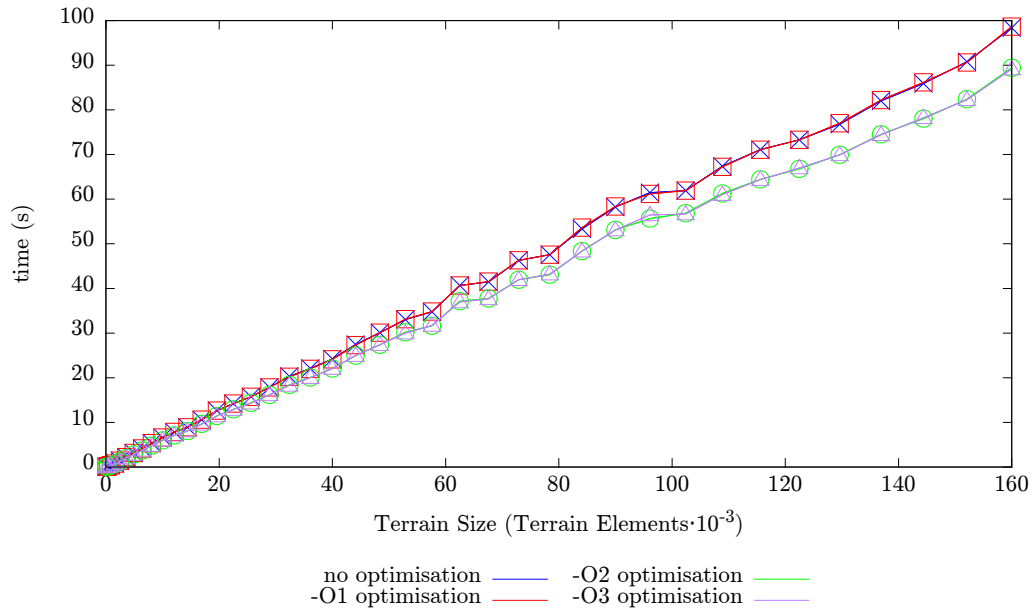


Figure 2: Graph showing pgi execution times

3.2 Performance Characteristics

4 Conclusion

They are both well fast

References

- [1] GNU. *Optimize Options* <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> Accessed: 14-11-2016
- [2] The Portland Group *PGI Compiler User's Guide for Intel 64 and AMD64C PUs* <http://www.pgroup.com/doc/pgiug-x64.pdf> Accessed: 14-11-2016

A Details of Processor and Compilers

A.1 Processor

```
Architecture:      x86_64
CPU op-mode(s):    32-bit, 64-bit
Byte Order:        Little Endian
CPU(s):            4
On-line CPU(s) list: 0-3
Thread(s) per core: 1
Core(s) per socket: 4
Socket(s):         1
NUMA node(s):      1
Vendor ID:         GenuineIntel
CPU family:        6
Model:             60
Model name:        Intel(R) Core(TM) i5-4570S CPU @ 2.90GHz
Stepping:          3
CPU MHz:           1200.667
BogoMIPS:          5786.47
Virtualization:    VT-x
L1d cache:         32K
L1i cache:         32K
L2 cache:          256K
L3 cache:          6144K
NUMA node0 CPU(s): 0-3
```

A.2 PGI C Compiler

```
pgcc 15.5-0 64-bit target on x86-64 Linux -tp haswell
The Portland Group - PGI Compilers and Tools
```

A.3 GNU C Compiler

```
gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-4)
```